

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1419

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO3: *Analyze syntax-directed translation method using synthesized and inherited attributes to generate a Parser Tree. (BL4)*

Assessment Tool Weightage: 40%

QUESTIONS:5

Mode: In Class

Total Marks:100

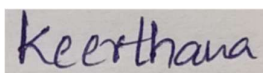
Hours : 60 mts

Annexure A

EXPERIMENTS

*I P Keerthana (192511395) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



Q. No	Understanding of Concepts (25)	Experimental Design (30)	Use of Tools (20)	Analysis of Results (15)	Documentation (10)	Total Score (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	
5	/ 4	/ 4	/ 4	/ 4	/ 4	
OVERALL RUBRICS VALUE						
	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 20	/ 20	/ 25	/ 20	/ 15	/ 100

Annexure A

EXPERIMENTS

Q1.

You are designing a compiler module that evaluates arithmetic expressions using syntax-directed definitions (SDD). The system must:

- (i) Accept arithmetic expressions with operators +, -, *, / and parentheses
- (ii) Construct a syntax tree and evaluate expressions using synthesized attributes
- (iii) Respect operator precedence and associativity rules
- (iv) Produce the final computed value of the expression

Demonstrate how SDD rules are applied for evaluating expressions and explain the bottom-up evaluation process using annotated parse trees.

Q2.

In a compiler, intermediate code generation often uses postfix expressions for efficient evaluation. The system must:

- (i) Accept a postfix expression as input
- (ii) Use a stack-based approach to evaluate the expression
- (iii) Demonstrate bottom-up computation of intermediate results
- (iv) Display the final evaluated result

Write a C program to evaluate postfix expressions and explain how this process relates to S-attributed definitions in compilers.

Q3.

You are developing a compiler component that uses L-attributed definitions for expression evaluation. The system must:

- (i) Accept arithmetic expressions such as $a + b * c$
- (ii) Use inherited attributes to pass information during parsing
- (iii) Ensure correct operator precedence
- (iv) Perform evaluation using a top-down approach

Write a C program that simulates L-attributed evaluation and explain how inherited attributes are used to propagate values during parsing.

Q4.

A compiler's semantic analyzer must check type compatibility between expressions. The system should:

- (i) Accept two type expressions such as int, float, int*, char*
- (ii) Compare the types for equivalence
- (iii) Handle both basic and pointer types
- (iv) Display whether the types are equivalent or not

Write a C program to perform type equivalence checking and explain how compilers use this mechanism during semantic analysis.

Q5.

You are designing a semantic analyzer for a compiler that detects type errors in arithmetic expressions. The system must:

- (i) Accept operands along with their types
 - (ii) Validate operations such as +, -, , /
 - (iii) *Detect invalid combinations such as int + char*
 - (iv) Report whether the expression is valid or contains a type error
- Write a C program to perform type checking and explain how semantic analysis ensures correctness in program execution.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1. Code

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #include <ctype.h>
4. #include <string.h>
5.
6. char expression[100];
7. int pos = 0;
8.
9. /* Syntax Tree Node */
10. struct Node
11. {
12.     char data;
13.     int value;
14.     struct Node *left;
15.     struct Node *right;
16. };
17.
18. /* Create node */
19. struct Node* createNode(char data, int value)
20. {
21.     struct Node *newNode;
22.
23.     newNode = (struct Node*)malloc(sizeof(struct Node));
24.
25.     newNode->data = data;
26.     newNode->value = value;
27.     newNode->left = NULL;
28.     newNode->right = NULL;
29.
30.     return newNode;
31. }
32.
33. /* Function declarations */
34. struct Node* E();
35. struct Node* T();
36. struct Node* F();
37.
38. /* E -> T { +T | -T } */
39. struct Node* E()
40. {
41.     struct Node *left, *right, *newNode;
42.     char op;
43.
44.     left = T();
45.
46.     while (expression[pos] == '+' || expression[pos] == '-')
47.     {
```

```

48.         op = expression[pos];
49.         pos++;
50.
51.         right = T();
52.
53.         newNode = createNode(op, 0);
54.         newNode->left = left;
55.         newNode->right = right;
56.
57.         left = newNode;
58.     }
59.
60.     return left;
61.}
62.
63./* T -> F { *F | /F } */
64.struct Node* T()
65.{
66.    struct Node *left, *right, *newNode;
67.    char op;
68.
69.    left = F();
70.
71.    while (expression[pos] == '*' || expression[pos] == '/')
72.    {
73.        op = expression[pos];
74.        pos++;
75.
76.        right = F();
77.
78.        newNode = createNode(op, 0);
79.        newNode->left = left;
80.        newNode->right = right;
81.
82.        left = newNode;
83.    }
84.
85.    return left;
86.}
87.
88./* F -> (E) | number */
89.struct Node* F()
90.{
91.    struct Node *node;
92.    int number = 0;
93.
94.    if (expression[pos] == '(')
95.    {

```

```

96.         pos++;
97.
98.         node = E();
99.
100.             if (expression[pos] == ')')
101.                 pos++;
102.         }
103.     else
104.     {
105.         while (isdigit(expression[pos]))
106.         {
107.             number = number * 10 + (expression[pos] - '0');
108.             pos++;
109.         }
110.
111.         node = createNode('N', number);
112.     }
113.
114.     return node;
115. }
116.
117. /* Evaluate tree using synthesized attributes */
118. int evaluate(struct Node *root)
119. {
120.     int leftValue, rightValue;
121.
122.     if (root == NULL)
123.         return 0;
124.
125.     /* Number */
126.     if (root->data == 'N')
127.         return root->value;
128.
129.     /* Bottom-up evaluation */
130.     leftValue = evaluate(root->left);
131.     rightValue = evaluate(root->right);
132.
133.     switch (root->data)
134.     {
135.         case '+':
136.             root->value = leftValue + rightValue;
137.             break;
138.
139.         case '-':
140.             root->value = leftValue - rightValue;
141.             break;
142.
143.         case '*':

```

```

144.         root->value = leftValue * rightValue;
145.         break;
146.
147.     case '/':
148.         if (rightValue == 0)
149.         {
150.             printf("Division by zero is not allowed.\n");
151.             exit(1);
152.         }
153.
154.         root->value = leftValue / rightValue;
155.         break;
156.     }
157.
158.     return root->value;
159. }
160.
161. /* Print syntax tree */
162. void printTree(struct Node *root, int space)
163. {
164.     int i;
165.
166.     if (root == NULL)
167.         return;
168.
169.     /* Right subtree */
170.     printTree(root->right, space + 5);
171.
172.     for (i = 0; i < space; i++)
173.         printf(" ");
174.
175.     if (root->data == 'N')
176.         printf("%d", root->value);
177.     else
178.         printf("%c", root->data);
179.
180.     printf("\n");
181.
182.     /* Left subtree */
183.     printTree(root->left, space + 5);
184. }
185.
186. /* Print annotated tree */
187. void printAnnotatedTree(struct Node *root, int space)
188. {
189.     int i;
190.
191.     if (root == NULL)

```

```

192.         return;
193.
194.     printAnnotatedTree(root->right, space + 5);
195.
196.     for (i = 0; i < space; i++)
197.         printf(" ");
198.
199.     if (root->data == 'N')
200.         printf("%d[%d]\n", root->value, root->value);
201.     else
202.         printf("%c[%d]\n", root->data, root->value);
203.
204.     printAnnotatedTree(root->left, space + 5);
205. }
206.
207. /* Main */
208. int main()
209. {
210.     struct Node *root;
211.     int result;
212.
213.     /* ----- */
214.     /* 1. Accept arithmetic expression */
215.     /* ----- */
216.
217.     printf("Enter arithmetic expression: ");
218.     scanf("%99s", expression);
219.
220.     printf("\n1. Expression Accepted:");
221.     printf("\n   %s\n", expression);
222.
223.     /* ----- */
224.     /* 2. Construct syntax tree */
225.     /* ----- */
226.
227.     root = E();
228.
229.     printf("\n2. Syntax Tree:\n\n");
230.     printTree(root, 0);
231.
232.     /* ----- */
233.     /* 3. Evaluate using synthesized attributes */
234.     /* ----- */
235.
236.     result = evaluate(root);
237.
238.     printf("\n3. Annotated Syntax Tree:\n\n");
239.     printAnnotatedTree(root, 0);

```



```

240.
241.      /* ----- */
242.      /* 4. Final result */
243.      /* ----- */
244.
245.      printf("\n4. Final Result = %d\n", result);
246.
247.      printf("\nBottom-Up Evaluation:\n");
248.      printf("Leaf nodes -> Operators -> Root\n");
249.
250.      return 0;
251.  }

```

Output:

Console

I/O

AI Agent

2 ms | 4.6 MB

STDIN

2+3*4

Output:

1. Expression Accepted:

2+3*4

2. Syntax Tree:

4

*

3

+

2

3. Annotated Syntax Tree:

4[4]

*[12]

3[3]

+[14]

2[2]

4. Final Result = 14

Bottom-Up Evaluation:

Leaf nodes -> Operators -> Root

2. Code

```
1. #include <stdio.h>
2. #include <ctype.h>
3.
4. int stack[100];
5. int top = -1;
6.
7. /* Push element into stack */
8. void push(int value)
9. {
10.     stack[++top] = value;
11. }
12.
13. /* Pop element from stack */
14. int pop()
15. {
16.     return stack[top--];
17. }
18.
19. /* Display stack */
20. void displayStack()
21. {
22.     int i;
23.
24.     printf("Stack: ");
25.
26.     for (i = 0; i <= top; i++)
27.     {
28.         printf("%d ", stack[i]);
29.     }
30.
31.     printf("\n");
32. }
33.
34. int main()
35. {
36.     char postfix[100];
37.     int i;
38.     int a, b, result;
39.
40.     /* ----- */
41.     /* 1. Accept postfix expression */
42.     /* ----- */
43.
44.     printf("Enter postfix expression: ");
45.     scanf("%99s", postfix);
46.
47.     printf("\n1. Postfix Expression Accepted:\n");
```

```

48.     printf("    %s\n", postfix);
49.
50.     /* ----- */
51.     /* 2. Stack-based evaluation */
52.     /* ----- */
53.
54.     printf("\n2. Stack-Based Evaluation:\n\n");
55.
56.     for (i = 0; postfix[i] != '\0'; i++)
57.     {
58.         /* If operand, push into stack */
59.         if (isdigit(postfix[i]))
60.         {
61.             push(postfix[i] - '0');
62.
63.             printf("Read operand %c -> Push %d\n",
64.                 postfix[i], postfix[i] - '0');
65.
66.             displayStack();
67.         }
68.
69.         /* If operator, pop two operands */
70.         else
71.         {
72.             b = pop();
73.             a = pop();
74.
75.             switch (postfix[i])
76.             {
77.                 case '+':
78.                     result = a + b;
79.                     break;
80.
81.                 case '-':
82.                     result = a - b;
83.                     break;
84.
85.                 case '*':
86.                     result = a * b;
87.                     break;
88.
89.                 case '/':
90.                     if (b == 0)
91.                     {
92.                         printf("Error: Division by zero\n");
93.                         return 1;
94.                     }
95.

```

```

96.         result = a / b;
97.         break;
98.
99.     default:
100.         printf("Invalid operator\n");
101.         return 1;
102.     }
103.
104.     printf("Operator %c: %d %c %d = %d\n",
105.         postfix[i], a, postfix[i], b, result);
106.
107.     /* Push intermediate result */
108.     push(result);
109.
110.     displayStack();
111.     printf("\n");
112. }
113. }
114.
115.     /* ----- */
116.     /* 3. Bottom-up computation */
117.     /* ----- */
118.
119.     printf("3. Bottom-Up Computation:\n");
120.     printf("    Operands -> Intermediate Results -> Final\n");
121.     printf("Result\n");
122.     /* ----- */
123.     /* 4. Final result */
124.     /* ----- */
125.
126.     result = pop();
127.
128.     printf("\n4. Final Evaluated Result = %d\n", result);
129.
130.     return 0;
131. }

```

Output:

Console I/O AI Agent 2 ms | 4.5 MB

STDIN

23*54*+

Output:

1. Postfix Expression Accepted:

23*54*+

2. Stack-Based Evaluation:

Read operand 2 -> Push 2

Stack: 2

Read operand 3 -> Push 3

Stack: 2 3

Operator *: 2 * 3 = 6

Stack: 6

Read operand 5 -> Push 5

Stack: 6 5

Read operand 4 -> Push 4

Stack: 6 5 4

Operator *: 5 * 4 = 20

Stack: 6 20

Operator +: 6 + 20 = 26

Stack: 26

3. Bottom-Up Computation:

Operands -> Intermediate Results -> Final Result

4. Final Evaluated Result = 26

3. Code

```
1. #include <stdio.h>
2. #include <ctype.h>
3.
4. char expression[100];
5. int pos = 0;
6.
7. /* Values of variables */
8. int a, b, c;
9.
10. /* Get value of a variable */
11. int getValue(char ch)
12. {
13.     if (ch == 'a')
14.         return a;
15.     else if (ch == 'b')
16.         return b;
17.     else if (ch == 'c')
18.         return c;
19.
20.     return 0;
21. }
22.
23. /*
24.  F -> id
25.  Returns the value of the variable
26. */
27. int F()
28. {
29.     char variable;
30.     int value;
31.
32.     variable = expression[pos];
33.     pos++;
34.
35.     value = getValue(variable);
36.
37.     printf("F: Variable %c has value %d\n",
38.           variable, value);
39.
40.     return value;
41. }
42.
43. /*
44.  T -> F { * F }
45.  Handles multiplication
46. */
47. int T()
```

```

48.{
49.    int inheritedValue;
50.    int value;
51.
52.    /* Inherited value passed from T to F */
53.    inheritedValue = F();
54.
55.    value = inheritedValue;
56.
57.    while (expression[pos] == '*')
58.    {
59.        pos++;
60.
61.        /* Pass current value as inherited information */
62.        value = value * F();
63.
64.        printf("T: Intermediate value = %d\n", value);
65.    }
66.
67.    return value;
68.}
69.
70./*
71.  E -> T { + T }
72.  Handles addition
73.*/
74.int E()
75.{
76.    int inheritedValue;
77.    int value;
78.
79.    /* Get first term */
80.    inheritedValue = T();
81.
82.    value = inheritedValue;
83.
84.    while (expression[pos] == '+')
85.    {
86.        pos++;
87.
88.        /* Pass current value to next term */
89.        value = value + T();
90.
91.        printf("E: Intermediate value = %d\n", value);
92.    }
93.
94.    return value;
95.}

```

```

96.
97. int main()
98. {
99.     int result;
100.
101.     /* ----- */
102.     /* 1. Accept arithmetic expression */
103.     /* ----- */
104.
105.     printf("Enter expression (example a+b*c): ");
106.     scanf("%99s", expression);
107.
108.     printf("\n1. Expression Accepted:\n");
109.     printf("    %s\n", expression);
110.
111.     /* ----- */
112.     /* Read variable values */
113.     /* ----- */
114.
115.     printf("\nEnter value of a: ");
116.     scanf("%d", &a);
117.
118.     printf("Enter value of b: ");
119.     scanf("%d", &b);
120.
121.     printf("Enter value of c: ");
122.     scanf("%d", &c);
123.
124.     /* ----- */
125.     /* 2. L-attributed evaluation */
126.     /* ----- */
127.
128.     printf("\n2. L-Attributed Evaluation:\n\n");
129.
130.     result = E();
131.
132.     /* ----- */
133.     /* 3. Operator precedence */
134.     /* ----- */
135.
136.     printf("\n3. Operator Precedence:\n");
137.     printf("    Multiplication (*) is evaluated before addition
    (+).\n");
138.
139.     /* ----- */
140.     /* 4. Final result */
141.     /* ----- */
142.

```



```
143.         printf("\n4. Final Result = %d\n", result);
144.
145.         return 0;
146.     }
```

Output:

Console

I/O

AI Agent

2 ms | 4.5 MB

STDIN

a+b*c

10

5

2

Output:

Enter expression (example a+b*c):

1. Expression Accepted:

a+b*c

Enter value of a: Enter value of b: Enter value of c:

2. L-Attributed Evaluation:

F: Variable a has value 10

F: Variable b has value 5

F: Variable c has value 2

T: Intermediate value = 10

E: Intermediate value = 20

3. Operator Precedence:

Multiplication (*) is evaluated before addition (+).

4. Final Result = 20

4. Code

```
1. #include <stdio.h>
2. #include <string.h>
3.
4. /* Function to check type equivalence */
5. int checkEquivalent(char type1[], char type2[])
6. {
7.     /* If both types are exactly the same */
8.     if (strcmp(type1, type2) == 0)
9.         return 1;
10.
11.     return 0;
12. }
13.
14. int main()
15. {
16.     char type1[20], type2[20];
17.
18.     /* ----- */
19.     /* 1. Accept two type expressions */
20.     /* ----- */
21.
22.     printf("Enter first type: ");
23.     scanf("%19s", type1);
24.
25.     printf("Enter second type: ");
26.     scanf("%19s", type2);
27.
28.     printf("\n1. Types Accepted:\n");
29.     printf("    First Type = %s\n", type1);
30.     printf("    Second Type = %s\n", type2);
31.
32.     /* ----- */
33.     /* 2. Compare types */
34.     /* ----- */
35.
36.     printf("\n2. Comparing Types...\n");
37.
38.     /* ----- */
39.     /* 3. Handle basic and pointer types */
40.     /* ----- */
41.
42.     printf("\n3. Type Categories:\n");
43.
44.     if (type1[strlen(type1) - 1] == '*')
45.         printf("    First type is a pointer type.\n");
46.     else
```

```

47.         printf("    First type is a basic type.\n");
48.
49.     if (type2[strlen(type2) - 1] == '*')
50.         printf("    Second type is a pointer type.\n");
51.     else
52.         printf("    Second type is a basic type.\n");
53.
54.     /* ----- */
55.     /* 4. Display equivalence result */
56.     /* ----- */
57.
58.     printf("\n4. Type Equivalence Result:\n");
59.
60.     if (checkEquivalent(type1, type2))
61.         printf("    Types are EQUIVALENT.\n");
62.     else
63.         printf("    Types are NOT EQUIVALENT.\n");
64.
65.     return 0;
66. }

```

Output:

Console	I/O	AI Agent	2 ms
---------	-----	----------	------

STDIN

int

float

Output:

```

Enter first type: Enter second type:
1. Types Accepted:
   First Type = int
   Second Type = float

2. Comparing Types...

3. Type Categories:
   First type is a basic type.
   Second type is a basic type.

4. Type Equivalence Result:
   Types are NOT EQUIVALENT.

```

5. Code

```
1. #include <stdio.h>
2. #include <string.h>
3.
4. /* Function to check whether a type is valid */
5. int validType(char type[])
6. {
7.     if (strcmp(type, "int") == 0 ||
8.         strcmp(type, "float") == 0 ||
9.         strcmp(type, "char") == 0)
10.    {
11.        return 1;
12.    }
13.
14.    return 0;
15.}
16.
17./* Function to perform type checking */
18.int typeCheck(char type1[], char op, char type2[])
19.{
20.    /*
21.        For this simple compiler model:
22.        int + int      -> valid
23.        float + float  -> valid
24.        int * int      -> valid
25.        float / float  -> valid
26.
27.        int + char     -> invalid
28.        float * char   -> invalid
29.        char + char    -> invalid
30.    */
31.
32.    if (!validType(type1) || !validType(type2))
33.        return 0;
34.
35.    /* char is not allowed in arithmetic operations */
36.    if (strcmp(type1, "char") == 0 ||
37.        strcmp(type2, "char") == 0)
38.    {
39.        return 0;
40.    }
41.
42.    /* Both operands must have the same type */
43.    if (strcmp(type1, type2) != 0)
44.    {
45.        return 0;
46.    }
47.}
```

```

48.  /* Check valid arithmetic operators */
49.  if (op == '+' || op == '-' || op == '*' || op == '/')
50.      return 1;
51.
52.  return 0;
53.}
54.
55.int main()
56.{
57.    char operand1[20], operand2[20];
58.    char type1[20], type2[20];
59.    char op;
60.
61.    /* ----- */
62.    /* 1. Accept operands and types */
63.    /* ----- */
64.
65.    printf("Enter first operand: ");
66.    scanf("%19s", operand1);
67.
68.    printf("\nEnter type of first operand: ");
69.    scanf("%19s", type1);
70.
71.    printf("\nEnter operator (+, -, *, /): ");
72.    scanf(" %c", &op);
73.
74.    printf("\nEnter second operand: ");
75.    scanf("%19s", operand2);
76.
77.    printf("\nEnter type of second operand: ");
78.    scanf("%19s", type2);
79.
80.    printf("\n1. Input Accepted:\n");
81.    printf("    %s (%s) %c %s (%s)\n",
82.        operand1, type1, op, operand2, type2);
83.
84.    /* ----- */
85.    /* 2. Validate operator */
86.    /* ----- */
87.
88.    printf("\n2. Checking Arithmetic Operation...\n");
89.
90.    if (op == '+' || op == '-' || op == '*' || op == '/')
91.        printf("    Operator %c is valid.\n", op);
92.    else
93.    {
94.        printf("    Invalid operator.\n");
95.        return 0;

```

```

96.     }
97.
98.     /* ----- */
99.     /* 3. Type checking */
100.    /* ----- */
101.
102.    printf("\n3. Type Checking:\n");
103.
104.    if (typeCheck(type1, op, type2))
105.    {
106.        printf("    Types are compatible.\n");
107.    }
108.    else
109.    {
110.        printf("    Type Error: Incompatible operand types.\n");
111.    }
112.
113.    /* ----- */
114.    /* 4. Final result */
115.    /* ----- */
116.
117.    printf("\n4. Final Result:\n");
118.
119.    if (typeCheck(type1, op, type2))
120.        printf("    Expression is VALID.\n");
121.    else
122.        printf("    Expression contains a TYPE ERROR.\n");
123.
124.    return 0;
125. }

```

Output:

Console	I/O	AI Agent
---------	-----	----------

```
STDIN
x
float
*
y
float
```

Output:

```
Enter first operand:
Enter type of first operand:
Enter operator (+, -, *, /):
Enter second operand:
Enter type of second operand:
1. Input Accepted:
  x (float) * y (float)

2. Checking Arithmetic Operation...
  Operator * is valid.

3. Type Checking:
  Types are compatible.

4. Final Result:
  Expression is VALID.
```